

INFO0027: File Deduplication Service Project

Group size: 2

Deadline: 2026-05-15

1 Situation

You are working on an online file sharing and collaboration service provider (similar to Nextcloud).

Recently, the business analysts have received a customer demand: the platform incurs high storage costs because users upload identical files:

User#898 issue

Hello, I have a lot of photos and documents that I upload to the platform. I often end up uploading the same file multiple times, either because I forget I've already uploaded it or because I want to share it with different people. This is really frustrating because it takes up a lot of my storage space and makes it harder to manage my files.

The product team has requested a new feature to detect and deduplicate these files.

This project is not happening in a vacuum. You have to satisfy multiple stakeholders with different timelines and requirements:

- **The Frontend Team** is already building the UI and have provided you the JSON requests you will need to fulfil.
- **The Legacy Core:** You already have a highly optimised, low-level deduplication library written in C, as part of your algorithmic project (if not, contact us we will provide the executable). Your service can use it, but should also be able to seamlessly swap to a pure Java/Kotlin implementation.
- **The Storage Team** is observing your progress. They plan to integrate your core logic directly into the upload pipeline. However, they have slightly different needs. They will want to verify files on-the-fly during the upload process, and warn users about potential duplicate as to avoid unnecessary uploads.
- **The Photography Clients** have caught wind of this feature and requested a variation: instead of exact deduplication, they want to group **similar** photos.

Your task is to design a flexible, maintainable architecture that can handle these diverging needs and implement it. You will therefore create a new deduplication service from the ground up, which adheres to these requirements and operate with the other services (their interfaces are provided).

2 Requirements

Your solution must fulfil the following architectural and functional requirements:

1. You must cleanly model the core entities of the system. Use appropriate interfaces to decouple the domain from the specific deduplication logic.

2. The frontend team has provided a `FrontendGate` interface that your system must implement. It uses JSON encoded strings to communicate. It has two methods, `String accept(String jsonRequest)` and `Stream<String> acceptStream(String jsonRequest)`. The first accept a request and returns a full answer (see following examples), while the second accept the same requests, but answer with a stream of groups represented as JSON arrays of virtual file paths encoded as strings.

In addition to the following response examples, the `FrontendGate` interface [JavaDoc](#) provides examples with error responses.

Example of request

```
{
  "action": "scan_duplicates",
  "scan_type": "exact",
  "path": "/documents",
  "user": "alice"
}
```

Example of request

```
{
  "action": "scan_duplicates",
  "scan_type": "similar",
  "path": "/images",
  "user": "alice"
}
```

Example of answer

```
{
  "status": "success",
  "groups": [
    [
      "/dubrovnik_0001.cr2",
      "/Dubrovnik-1.jpeg",
      "/backup/dubrovnik_0001.jpg"
    ]
  ]
}
```

Example of streaming answer

An item in the stream:

```
["/notes.md", "/Documents/notes-2025-04-18.txt"]
```

Second item in the stream:

```
["/hello.txt", "/.cache/hello.txt", "/archive/helloworld.txt"]
```

3. You must support different deduplication engines that can be swapped easily:

1. A pure Java/Kotlin implementation for exact duplicates of files.

2. *Facultatively*, an exact deduplication based on interoperability with your existing C library using the following signatures:

```
typedef struct magic *FILEDEDUP;  
FILEDEDUP FDEInit(void);  
int FDCheck(FILEDEDUP fd, char *filepath);  
char **FDDump(FILEDEDUP fd, int *length);
```

3. The similarity based deduplication for the photography use-case. Potentially for any kind of file in the future.
4. Plan your interfaces to support the Storage team's upcoming needs. They need to deduplicate files independently of users, across all the storage space to allow at most one copy of any files.
5. You will be provided with a `VirtualFileSystem` interface that enables access to storage. All your deduplication logic must run against this virtual file system API.

3 Evaluation

This project counts towards 20% of your final mark. In this project, you will be evaluated on:

1. **Architecture and UML Class Diagrams:** Your report will discuss your *software architecture* with references to a *UML Class Diagram* of your solution.
2. **Software Patterns:** Your solution should use relevant software design patterns. Your report identifies these patterns, provides rationales for their application within your architecture, and discusses any potential trade-offs or limitations considered. Aim for effective use of patterns without unnecessary complexity.
3. **Functional Suitability:** Your solution must correctly implement all functional requirements. Automated tests will be used.
4. **Maintainability** your code maintainability will be evaluated based on ISO/IEC 25010:2024 *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* ([edu.nbn.be](https://www.edunbn.be)).
5. **Quality assurance/Test Plan:** Your report should describe how you envision quality for this software. Choose at least 3 relevant main quality characteristics and their relevant sub-characteristics. How are they applied for this project, how do you plan to track them, and ensure they are met. Describe your test plan, include any test implementation, and as annex you test logs.

4 Deliverables: Report and Implementation

Projects must be submitted before 2026-05-15, 23:59, by groups of 2, on [Gradescope](https://www.gradescope.com) with the course entry code `J6Y4N3`. After this time, a penalty will be applied to late submissions. This penalty is calculated as a deduction of 2^{N-1} marks (where N is the number of started days after the deadline).

Your submission archive should contain your **implementation** of the File Deduplication Service and your **report**.

You are required to declare the use of (generative) AI for the production of your deliverables in line with [CEUR-WS Policy on AI-Assisting Tools](#). The Declaration on Generative AI, which is a separate section or paragraph in your report, contains either “The author(s) have not employed any Generative AI tools” or, by using the activity taxonomy, something along the lines of “During the preparation of this work, the author(s) used X-GPT-4 and Grammarly for grammar and spelling check. Further, the author(s) used X-AI-Dev for code completion, and test cases generation. After using these tools/services, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the publication’s content.” If any intended use of (generative) AI does not appear in the taxonomy, then disclose this in the statement as well.

4.1 Report

Your report should be a PDF file at the root of your submission. It should be a total (except annexes) maximum of 6 pages in English, in a reasonable layout.

- Describe your application’s main components and their interactions (this does not substitute for code documentation).
- Include a *UML Class Diagram* that visually represents the structure of your software design.
- Identify the *design patterns* you implemented. For each, explain its purpose and trade-offs in your design.
- Unveil your test plan and quality assurance strategies.
- Include your design process, including any challenges you faced and how you overcame them.
- Discuss limitations of your design and implementation, including any known bugs or areas for improvement.

Think of this report as a helpful guide for the next person who will need to update or add features to this project based on future requirements.

4.2 Implementation

You can choose to implement the project with **Java** or **Kotlin**. Choose whichever you dare to use.

Your application will run on Eclipse Temurin JDK 25 LTS, with Kotlin 2.3.20, Maven, and Gradle 9.4.0.

You can use a library for JSON handling and image similarity, but in general, try to minimise dependencies.

If you would like to use additional libraries, ask on the discussion system of eCampus. To embed your libraries inside the jar and avoid classpath issues, you may find the [Shadow plugin](#) (for Gradle) or the Maven Assembly Plugin useful to create self-contained “fat” JARs.

To facilitate automated testing, **we provide a set of standard interfaces your system must use**. These interfaces are hosted on our [institutional GitLab Package Registry](#). You must add this repository and the dependency to your build configuration. The source code of these interfaces is also provided in that same repository [info0027-filededup-interfaces](#). You may need to implement those interfaces yourself for testing.

4.2.1 Automated Testing & Build Process

Our automated test suite will build your project, find your compiled “Fat JAR”, and run integration tests against it. To achieve this, your build script must be capable of producing a standalone JAR file named `Deduplicator.jar` at the root of your project upon execution.

ServiceLoader Configuration: Because you are free to name your implementing classes however you like, our test suite will use Java’s [ServiceLoader](#) to dynamically discover and instantiate your implementation of the `FileDeduplicationBootstrap` class.

To make your implementation discoverable, you must create a specific provider configuration file in your resources directory. This works identically for both Maven and Gradle projects.

1. Navigate to your resources folder: `src/main/resources/`
2. Create a directory structure named exactly `META-INF/services/`
3. Inside that directory, create a text file named exactly with the fully qualified name of the interface: `be.uliege.info0027.deduplication.FileDeduplicationBootstrap`
4. Inside this text file, write a single line containing the fully qualified name of your implementation class.

For example, if your Kotlin/Java class is `MyFrontendGateImpl`, the contents of `src/main/resources/META-INF/services/`
`be.uliege.info0027.deduplication.FileDeduplicationBootstrap` should be exactly:

`MyFileDeduplicationBootstrap`

During the build process, Maven or Gradle will automatically copy this `META-INF` folder into your generated `Deduplicator.jar`, allowing our test suite to load it natively without needing to know your internal architecture.

One week before the deadline, the list of allowed libraries and any modifications to the online test script will be frozen.

4.2.2 Native Library Interoperability

If you choose to implement the optional requirement of interoperating with the existing C library, here is a brief guide on how to set it up using *Java Panama*:

1. Ensure you have the C library compiled and accessible on your system (we can provide one, contact us). You will need the header files and the compiled shared library (e.g., `libfilededup.so` on Linux). You should be able to compile the shared library with `gcc -Wall -Wextra -pedantic -std=gnu11 -O3 -fPIC -shared src/filededup.c -o $(SHARED_LIB)`.

The flag `-fPIC` is crucial for creating a position-independent shared library.

2. In your Java/Kotlin code, you will need to define the native method signatures that correspond to the C functions. Use [jextract](#) to generate Java bindings for the C library. Either do it as a one off with the cli tool, or integrate it into your build process for gradle there exists [Gradle Jextract](#) plugin, we successfully used.
3. Then before calling any native methods, you will need to load the shared library. There is an example of how to proceed in the [Gradle Jextract Example](#) repository.

4.2.3 Comparing images in Java

To implement the similarity-based deduplication for the photography use case, you can use any suitable image processing library. Below is an example using [OpenCV](#) in Kotlin, which uses template matching to calculate a similarity score.

The logic is broken down into two logical steps: preprocessing the images to normalize their size and color space, and then comparing their matrices.

```
// Preprocess the image: standardize size and remove color variations
fun preprocessImage(filePath: String): Mat {
    val src = Imgcodecs.imread(filePath)
    val gray = Mat()

    // Convert to grayscale
    Imgproc.cvtColor(src, gray, Imgproc.COLOR_BGR2GRAY)
    src.release() // Release memory

    val resizedGray = Mat()
    // Resize to a fixed dimension (for example 256x256)
    Imgproc.resize(gray, resizedGray, Size(256.0, 256.0))
    gray.release()

    return resizedGray
}

// Compare two preprocessed image matrices
fun areImagesSimilar(leftMat: Mat, rightMat: Mat): Boolean {
    val res = Mat()

    // Use Template Matching with Normalized Correlation Coefficient
    Imgproc.matchTemplate(leftMat, rightMat, res, Imgproc.TM_CCOEFF_NORMED)
    val score = res.get(0, 0)[0]
    res.release()

    // TM_CCOEFF_NORMED returns 1.0 for exact matches.
    // 0.90 is a fine threshold for similarity.
    return score >= 0.90
}
```

Ask your questions early, and enjoy!